

3.1 Why infrastructure as code

- Now we are moving on to make the exact same application, but this time with IaC (Terraform).

The only difference we are going to make in this section of the project is NOT DELETING THINGS. Further on in the tickets it will start to tell you to delete all the work from week 2 and then everything you made in week 3.

BUT

We are not doing that; all we are going to do is name the new items different names. For example, for this Terraform version of the project we will be naming them

umar-coldchain-LinkDynamoDB#2

NO DELETING

Learning objective

What does "infrastructure as code" mean?

Infrastructure as Code (IaC) is the managing and provisioning of infrastructure through code instead of manual processes.

With IaC, configuration files are created that contain your infrastructure specifications, which makes it easier to edit and distribute configurations. It also ensures that you provision the same environment every time. By codifying and documenting your configuration specifications, IaC aids configuration management and helps you avoid undocumented, ad hoc configuration changes.

What problems does it solve? Think about: repeatability, being able to review a change before it happens, being able to undo it, and not having environments that quietly differ from each other

Infrastructure as Code solves a number of problems that come from setting up infrastructure manually.

- **Repeatability:** The same code can be used to create the same infrastructure again and again. This means you do not have to remember every manual step each time you create a new environment.
- **Review changes before they happen:** Tools such as Terraform can show you what changes will be made before they are applied. For example, `terraform plan` can show which AWS

resources will be created, changed, or deleted before anything actually happens.

- **Easier to undo changes:** Because the infrastructure code can be stored in Git, previous versions of the code are saved. If a change causes a problem, you can go back to an earlier version and redeploy the previous configuration.
- **Prevents environments from quietly becoming different:** When infrastructure is created manually, your development, testing, and production environments can slowly become different because someone may change a setting in one environment but forget to change the others. IaC helps keep them consistent because they are created from the same code.

Overall, IaC makes infrastructure **more consistent, repeatable, reviewable, and easier to manage**, while reducing mistakes caused by manually configuring resources.

Why does nobody here make production changes by clicking in the console?

It is a long and manual-intensive process and is prone to many human errors, e.g., misconfiguration. When projects become large with extensive infrastructure it is almost impossible to remember how and what you need to manage.

What do `terraform plan` and `terraform apply` each do, and why is plan the important one?

`terraform plan` = checks your Terraform files and shows you what Terraform intends to change. It tells you which resources will be created, updated, or destroyed, but it does not actually make those changes.

`terraform apply` = actually performs the changes. It creates, updates, or deletes the infrastructure so that it matches what is written in your Terraform code.

`terraform plan` = “**Show me what you are going to do.**”

`terraform apply` = “**Now actually do it.**”

In a real company environment, reviewing the plan is important because it helps prevent accidental infrastructure changes and allows other team members to review what will happen before it is deployed.



LUKE EXPLANATION = you want to be intensive with the plan, checking it over because once you click apply that's it — it's gone through. This is a big issue because if you didn't check the Terraform plan and it, for example, deletes an entire client's folder and you clicked apply, there's no getting that back.

Task

Write down every manual click you did in Week 2. Every table, function, route, topic, queue and permission

• DynamoDB

- Created the DynamoDB table:

```
1 | umar_coldchain_database
```

- Set the partition key to:

```
1 | branchId
```

- Set the sort key to:

```
1 | readingKey
```

- The `readingKey` stores:

```
1 | recordedAt#readingId
```

- Used the table to store temperature readings, their calculated `status`, and later `alertRaisedAt`.

• Main Lambda function

- Created the Lambda:

```
1 | umar-coldchain-LinkDynamoDB
```

- Used Node.js.
- Added the JavaScript code that:
 - handles GET and POST requests
 - validates readings
 - calculates `ok` or `excursion`
 - generates a `readingId`
 - saves readings to DynamoDB
 - checks for duplicate timestamps
 - publishes to SNS when the status is `excursion`
- Connected the Lambda to DynamoDB through its IAM execution role.
- Added permission for:

```
1 | dynamodb:PutItem
2 | dynamodb:GetItem
3 | dynamodb:Query
4 | dynamodb:UpdateItem
5 | dynamodb>DeleteItem
```

- Added permission for:

```
1 | sns:Publish
```

on:

```
1 | umar-coldchain-excursions
```

- Lambda also had permission to write logs to CloudWatch.

• API Gateway

- Created an HTTP API:

```
1 | umar-coldchain-api
```

- Connected API Gateway to:

```
1 | umar-coldchain-LinkDynamoDB
```

- Used Lambda proxy integration with payload format `2.0`.
- Created these three routes:

```
1 | GET /branches/{branchId}/readings
2 | POST /branches/{branchId}/readings
3 | GET /branches/{branchId}/readings/{recordedAt}
```

- Used the `$default` stage.
- Auto-deploy was enabled.
- API Gateway was given permission to invoke the Lambda.
- No API-key authentication was added, so this remains an area to revisit rather than something Terraform needs to reproduce right now.

• SNS

- Created the Standard SNS topic:

```
1 | umar-coldchain-excursions
```

- Configured the create-reading Lambda to publish a message to this topic only when:

```
1 | status = excursion
```

- The message contains information such as:

```
1 | branchId
2 | readingKey
3 | readingId
4 | recordedAt
5 | minTempC
6 | maxTempC
7 | status
```

• SQS main queue

- Created the Standard SQS queue:

```
1 | umar-coldchain-alert-queue
```

- Subscribed this queue to:

```
1 | umar-coldchain-excursions
```

- Enabled:

```
1 | Raw message delivery
```

- Set the queue visibility timeout to:

```
1 | 30 seconds
```

- Configured the queue to use a dead-letter queue.
- SNS/SQS permissions were configured so SNS could send messages to this queue.

• **Dead-letter queue**

- Created:

```
1 | umar-coldchain-alert-dlq
```

- Connected it to:

```
1 | umar-coldchain-alert-queue
```

- Configured the main queue's redrive policy so messages that repeatedly fail eventually move into the DLQ.
- We used a maximum receive count of approximately:

```
1 | 3 retries / failed processing attempts before redrive
```

- Tested it by deliberately throwing an error in the excursion-handler Lambda.

• **Excursion-handler Lambda**

- Created the second Lambda:

```
1 | umar-coldchain-excursion-handler
```

- Used Node.js.
- Set its Lambda timeout to:

```
1 | 5 seconds
```

- Connected:

```
1 | umar-coldchain-alert-queue
```

as an SQS trigger.

- Set the SQS event source mapping/trigger to enabled.
- Used a small batch size, effectively processing the alert messages individually during testing.
- Added the code that:

- reads an SQS message
- finds the matching DynamoDB reading
- sets `alertRaisedAt`
- deliberately throws when `forceFailure` is true
- detects duplicate processing
- logs successful alerts and failures

• Excursion-handler IAM permissions

- Attached SQS permissions equivalent to:

```
1 | AWSLambdaSQSQueueExecutionRole
```

- This gave the Lambda permission for actions such as:

```
1 | sqs:ReceiveMessage
2 | sqs:DeleteMessage
3 | sqs:GetQueueAttributes
```

- Added DynamoDB permission:

```
1 | dynamodb:UpdateItem
```

on:

```
1 | umar_coldchain_database
```

- Added CloudWatch logging permissions.

• Idempotency configuration

- Added a conditional DynamoDB update:

```
1 | ConditionExpression:
2 |   "attribute_not_exists(alertRaisedAt)"
```

- This means the first successful alert adds `alertRaisedAt` , but if the same SQS message arrives again, the original alert is not overwritten.
- Tested this by manually sending the same SQS message twice.

• CloudWatch

- Used CloudWatch log groups created for the Lambda functions.
- Checked logs for:

```
1 | umar-coldchain-LinkDynamoDB
2 | umar-coldchain-excursion-handler
```

- Used the logs to diagnose errors such as:

```
1 | AccessDeniedException
2 | ERR_MODULE_NOT_FOUND
3 | DELIBERATE_ALERT_FAILURE
```

- Verified successful alert processing through logs such as:

```
1 | ALERT RAISED
```

- **End-to-end manual connections**

- Manually connected the finished architecture as:

```
1 | Postman
2 |   ↓
3 | API Gateway
4 |   ↓
5 | umar-coldchain-LinkDynamoDB
6 |   ↓
7 | DynamoDB
8 |   |
8 |   └─ status = excursion
9 |       ↓
10 |      SNS
11 |       ↓
12 |      SQS
13 |       ↓
14 | Excursion Handler Lambda
15 |       ↓
16 |      DynamoDB
17 |       ↓
18 | alertRaisedAt
19 |
```

- Also configured the failure path:

```
1 | SQS
2 |   ↓
3 | Excursion Handler Lambda
4 |   ↓
5 | repeated failure
6 |   ↓
7 | Dead-Letter Queue
```

Basically every part of the system I have built can be made using Terraform code. “Any resource can be codified.”

⚠ Questions / Queries / Pushback